

Building And Tracing Java Expressions

Generated Jul 15, 2026 3:43 AM

What We'll Learn

By the end of this lesson, students can:

- Explain what a Java expression is
- Identify operands, operators, and results
- Build arithmetic, comparison, and logical expressions
- Follow Java's order of operations
- Trace expressions step by step
- Determine the final value stored in a variable
- Correct common expression errors

Words We'll Use

- Expression
- Operator
- Operand
- Evaluate
- Trace
- Result
- Precedence
- Parentheses
- Boolean
- Assignment

Before We Start

Consider the following expression:

```
int result = 10 + 5 * 2;
```

What value will be stored in `result`?

Will Java add first or multiply first?

Java follows specific rules when evaluating expressions.

Understanding these rules helps us predict the correct result of a program.

Let's Understand

An expression is a combination of values, variables, and operators that produces a result.

Example:

```
int total = 10 + 5;
```

In this statement:

- 10 and 5 are operands.
- + is the operator.
- 10 + 5 is the expression.
- 15 is the result.
- `total` stores the result.

Let's Look at Examples

Example 1: Trace an Arithmetic Expression

This example demonstrates operator precedence and parentheses.

Problem:

Create a Java program that evaluates two arithmetic expressions and displays their results.

Program:

```
public class ArithmeticExpression {  
    public static void main(String[] args) {  
        int firstResult = 10 + 5 * 2;  
        int secondResult = (10 + 5) * 2;  
  
        System.out.println("First Result: " + firstResult);  
        System.out.println("Second Result: " + secondResult);  
    }  
}
```

Output:

```
First Result: 20  
Second Result: 30
```

Trace of the first expression:

```
10 + 5 × 2  
10 + 10  
20
```

Trace of the second expression:

```
(10 + 5) × 2  
15 × 2  
30
```

Explanation:

In the first expression, multiplication is performed before addition.

In the second expression, the operation inside the parentheses is performed first.

Common mistake

Evaluating every expression strictly from left to right without considering operator precedence.

Mini-check

What is the result of the following expression?

```
int result = 6 + 4 * 3;
```

Example 2: Trace Changes in Variable Values

This example demonstrates how expressions update a variable.

Problem:

Create a Java program that stores a number and changes its value using several expressions.

Program:

```
public class VariableTracing {
    public static void main(String[] args) {
        int number = 10;

        System.out.println("Starting Value: " + number);

        number += 5;
        System.out.println("After Adding 5: " + number);

        number *= 2;
        System.out.println("After Multiplying by 2: " + number);

        number -= 4;
        System.out.println("After Subtracting 4: " + number);

        number /= 2;
        System.out.println("Final Value: " + number);
    }
}
```

Output:

```
Starting Value: 10
After Adding 5: 15
After Multiplying by 2: 30
After Subtracting 4: 26
Final Value: 13
```

Trace table:

Statement	Value of number
int number = 10;	10
number += 5;	15
number *= 2;	30
number -= 4;	26

Statement	Value of number
number /= 2;	13

Explanation:

Each statement uses the current value of number.

The new value replaces the previous value before the next statement is executed.

Common mistake

Using the original value 10 for every calculation.

Mini-check

What would the final value be if number -= 4; were removed?

Example 3: Trace a Logical Expression

This example demonstrates how comparison and logical expressions are evaluated.

Problem:

A student is eligible for recognition when:

- The average is at least 90
- The attendance is at least 85
- The student has no incomplete requirement

Create a program that evaluates the conditions.

Program:

```
public class RecognitionEligibility {
    public static void main(String[] args) {
        double average = 92.5;
        int attendance = 88;
        boolean hasIncompleteRequirement = false;

        boolean meetsAverage =
            average >= 90;

        boolean meetsAttendance =
            attendance >= 85;

        boolean hasCompleteRequirements =
            !hasIncompleteRequirement;

        boolean isEligible =
            meetsAverage
            && meetsAttendance
            && hasCompleteRequirements;

        System.out.println("Average Requirement: "
            + meetsAverage);

        System.out.println("Attendance Requirement: "
            + meetsAttendance);

        System.out.println("Requirements Complete: "
            + hasCompleteRequirements);

        System.out.println("Eligible: "
            + isEligible);
    }
}
```

Output:

```
Average Requirement: true
Attendance Requirement: true
Requirements Complete: true
Eligible: true
```

Trace:

```
average >= 90
92.5 >= 90
true
```

```
attendance >= 85
88 >= 85
true
```

```
!hasIncompleteRequirement
!false
true
```

Final expression:

```
true && true && true
true
```

Explanation:

Each comparison is evaluated first.

The logical NOT operator reverses the value of hasIncompleteRequirement.

The three results are then combined using &&.

All three conditions are true, so isEligible becomes true.

Common mistake

Using hasIncompleteRequirement directly instead of reversing it.

Incorrect:

```
boolean isEligible =
    meetsAverage
    && meetsAttendance
    && hasIncompleteRequirement;
```

Correct:

```
boolean isEligible =  
    meetsAverage  
    && meetsAttendance  
    && !hasIncompleteRequirement;
```

Mini-check

What will happen if attendance is changed to 80?

Let's Practice

Part A: Identify the Expression Parts

Study the statement:

```
int result = firstNumber + secondNumber;
```

Identify the following:

- Variables:
- Operator:
- Expression:
- Variable receiving the result:

Part B: Trace the Arithmetic Expressions

Determine the final result.

```
int result = 8 + 4 * 2;
```

Trace:

```
_____  
_____  
_____
```

Final result: ____

```
int result = (8 + 4) * 2;
```

Trace:

```
_____  
_____  
_____
```

Final result: ____

```
int result = 20 / 5 + 3;
```

Trace:

Final result: ____

```
int result = 10 + 17 % 5;
```

Trace:

Final result: ____

Part C: Trace the Variables

Complete the trace table.

```
int score = 20;
score += 5;
score *= 2;
score -= 10;
score++;
```

Statement	Value of score
int score = 20;	
score += 5;	
score *= 2;	
score -= 10;	
score++;	

Final value: ____

Part D: Trace the Boolean Expressions

Determine whether each result is true or false.

```
int age = 18;
boolean result = age >= 18;
```

Result: ____

```
int score = 70;
int attendance = 90;

boolean result =
    score >= 75 && attendance >= 80;
```

Result: ____

```
boolean hasID = false;
boolean hasPass = true;

boolean result = hasID || hasPass;
```

Result: ____

```
boolean isAbsent = true;

boolean result = !isAbsent;
```

Result: ____

Part E: Correct the Expressions

Correct each expression.

Incorrect:

```
double average =
    quiz + activity + exam / 3.0;
```

Correct:

Incorrect:

```
double result = 7 / 2;
```

Correct the expression so the result is 3.5.

Incorrect:

```
System.out.println("Total: " + 5 + 3);
```

Correct the expression so the output is Total: 8.

Now We Apply

Create a Java program that computes and traces the total cost of a purchase.

Use the following variables:

- Product name
- Product price
- Quantity
- Discount
- Delivery fee

Use this formula:

```
subtotal = price × quantity  
final total = subtotal - discount + delivery fee
```

The program must:

1. Declare and initialize all variables.
2. Compute the subtotal.
3. Compute the final total.
4. Display the value of each variable.
5. Display the subtotal.
6. Display the final total.
7. Add comments showing the order in which the expressions are evaluated.

Suggested program structure:

```
public class PurchaseExpression {  
    public static void main(String[] args) {  
        // Declare and initialize variables  
  
        // Compute the subtotal  
  
        // Compute the final total  
  
        // Display the results  
    }  
}
```

Challenge Task

Build a Grade Status Checker expression set: compute average, determine passing status, and display the result.

How Our Work Will Be Checked

- Correct expression structure
- Correct operators
- Correct use of parentheses
- Correct order of operations
- Correct variable values
- Correct result data types
- Correct tracing process
- Correct Java syntax
- Complete and clearly labeled output

Let's Reflect

Answer the following questions:

1. What is a Java expression?
2. What is the difference between an operator and an operand?
3. Why is operator precedence important?
4. How do parentheses affect an expression?
5. Why must variables be traced in order?
6. Why does integer division sometimes produce an unexpected result?
7. How are comparison expressions traced?
8. How are logical expressions traced?
9. How can a trace table help find program errors?

How We Show Learning

Students will demonstrate learning by:

- Identifying parts of Java expressions
- Translating formulas into Java syntax
- Building arithmetic expressions
- Building comparison expressions
- Building logical expressions
- Applying operator precedence
- Tracing variable values
- Completing trace tables
- Correcting incorrect expressions
- Explaining how a final result was produced

Student Activities

Identify the Data Type. Predict the Output. Fix the Operator Error. Complete the Java Expression. Simple Calculator Practice. Trace Boolean Results. Create variables for a student profile. Write expressions for area, total price, and average. Explain one expression step by step.

Resources / References

- Java Development Kit
- Java code editor or IDE
- Java operator precedence reference
- Java expression worksheet
- Variable trace table worksheet
- Boolean expression reference
- Sample Java programs